

SYSTEM DESIGN INTERVIEW · VOL.2 · CH.2

Nearby Friends

실시간 '근처 친구' 백엔드 시스템 설계

10억

전체 사용자

5 마일

'근처' 반경

334K/s

위치 업데이트 QPS

스터디 발표 자료 · 배성현

오늘의 발표 흐름

1

문제 이해 & 설계 범위

요구사항, 규모 추정(QPS)

2

개략 설계 (High-level)

WebSocket · Redis · Pub/Sub 구조

3

심화 설계 (Deep Dive)

컴포넌트별 스케일링 전략

4

확장 & 정리

랜덤 근처 사람, Erlang 대안, 요약

각 단계는 시스템 디자인 인터뷰의 표준 4단계 흐름을 따릅니다.

왜 별도 챕터인가? · Proximity Service와의 차이

Proximity Service

대상	상점·식당 등 비즈니스
위치 특성	정적 (거의 안 바뀜)
데이터	미리 색인(index) 가능
갱신	드물게 발생

Nearby Friends

대상	사람 (친구)
위치 특성	동적 (계속 이동)
데이터	실시간 스트림
갱신	수 초마다 발생

→ 핵심 차이는 '데이터의 동적 특성'. 위치가 끊임없이 바뀌므로 미리 색인할 수 없고, 실시간 전파가 관건.

실시간 전파 — 무슨 기술로 하나?

위치를 친구에게 '뿌리는' 일 = 메시징 문제. 여러 선택지 중 이 설계는 Redis Pub/Sub 을 쓴다 — 왜인지 미리 비교

이럴 때	선택	핵심 이유
실시간 · 유실 OK · 이력 불필요 (예: 위치 전파) ← 이 설계	Redis Pub/Sub	가볍고 초저지연. 구독자 없으면 드롭 → 부하 최소
재생·소비자그룹·영속성 필요 Redis는 유지하고 싶음	Redis Streams	ACK·consumer group 지원. 인프라 추가 없이 승격
대용량 이벤트 수집·재처리 파이프라인	Kafka	소비자 무관하게 영속. 오프셋 기반 재처리 가능
MSA 간 보장된 전달·복잡한 라우팅	RabbitMQ	AMQP 라우팅 + ACK/재시도로 전달 보장
운영 부담 최소 · 클라우드 통합	관리형 / Supabase	Cloud Pub/Sub, Supabase Realtime 등 BaaS

왜 Pub/Sub?

Nearby Friends 요구 = 실시간 + 유실 허용 + 이력 불필요. 딱 Pub/Sub의 특성. '전달 보장'이나 '재생'이 필요 없어서 더 무거운 Kafka/RabbitMQ가 과함.

이 책은 2021년 — 지금은 어떻게 바뀌었나?

'근처 친구' 개념은 살아있지만, 독립 기능이 아니라 세 갈래로 흡수됨 — 백엔드 코어(WS+Pub/Sub)는 거의 그대로

대표 사례

Instagram Map

(2025 출시)



- 이너서클과만 위치 공유
- 앱 켜져 있을 때만 갱신
- 1시간 자동 만료 (프라이버시)

책의 '10분 비활성 제외' → '1시간 자동 만료'로 진화

측	2021 (책)	2026 (현재)
제품 형태	독립 '근처 친구' 기능	SNS 지도 · 안전앱 · 이벤트 코드로 분화
프라이버시	10분 비활성 → 제외	1시간 자동 만료 · Ghost 모드 (기본값)
공간 인덱싱	Geohash (사각형)	H3(육각형) · S2 — 도심 밀집에 유리
위치 수집	30초 고정 보고	Sensor Fusion · UWB · 정지 시 절전
백엔드 코어	WS + Redis Pub/Sub	거의 동일 ✓ (여전히 정식)

핵심: 겉(제품·프라이버시·인덱싱)은 진화했지만 속(분산 실시간 fan-out)은 그대로 → 이 책을 지금 공부하는 이유.

요구사항 정리



기능 요구사항

- 모바일 앱에서 근처 친구 목록 표시
- 각 항목 = 거리 + 마지막 갱신 시각
- 목록은 수 초마다 갱신
- 위치 이력(history) 저장 (ML 활용)
- 10분 이상 비활성 친구는 목록에서 제외



비기능 요구사항

낮은 지연 (Low Latency)

친구 위치를 지연 없이 수신

신뢰성 (Reliability)

간헐적 데이터 손실은 허용

최종 일관성 (Eventual)

복제본 간 수 초 지연 OK

강한 일관성은 불필요 → Redis 캐시 + Pub/Sub 로 '빠르게, 대략 맞게' 전달하는 설계가 정당화됨

규모 추정 · Back-of-the-envelope

가정 (Assumptions)

DAU	1억 명
동시 접속	DAU의 10% = 1천만
위치 보고 주기	30초마다
평균 친구 수	400명
근처+온라인 친구	친구의 약 10%
페이지당 표시	20명

QPS 계산

① 위치 업데이트 QPS

1천만 (동시접속) ÷ 30초

≈ **334,000 /초**



② 전달(Fan-out) 부하

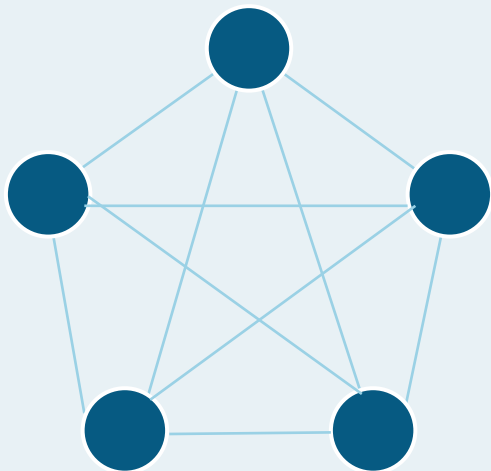
334K × 400 친구 × 10%

≈ **1,400만 전달 /초**

→ 받는 것보다 '뿌리는(fan-out)' 부하가 40배. 이 14M/s 를 어떻게 감당하느냐가 설계의 핵심 도전.

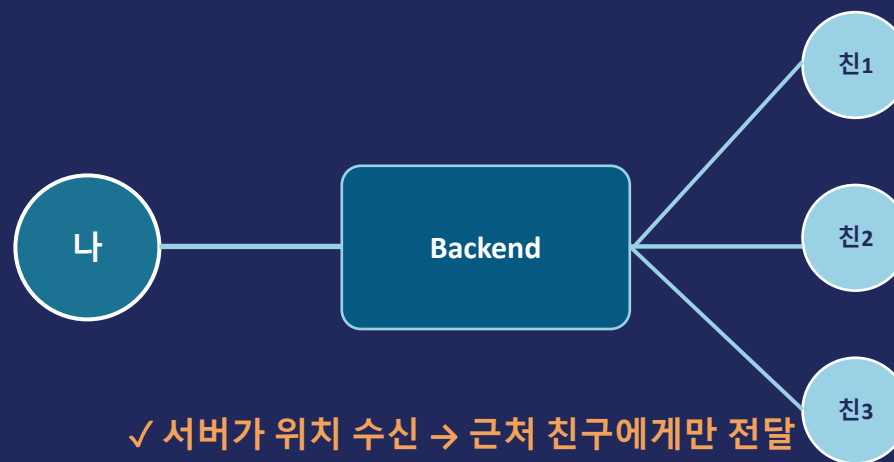
설계 방향 잡기

방법 A · P2P (또래 간 연결)



✗ 모바일은 연결 불안정 + 배터리 소모 큼

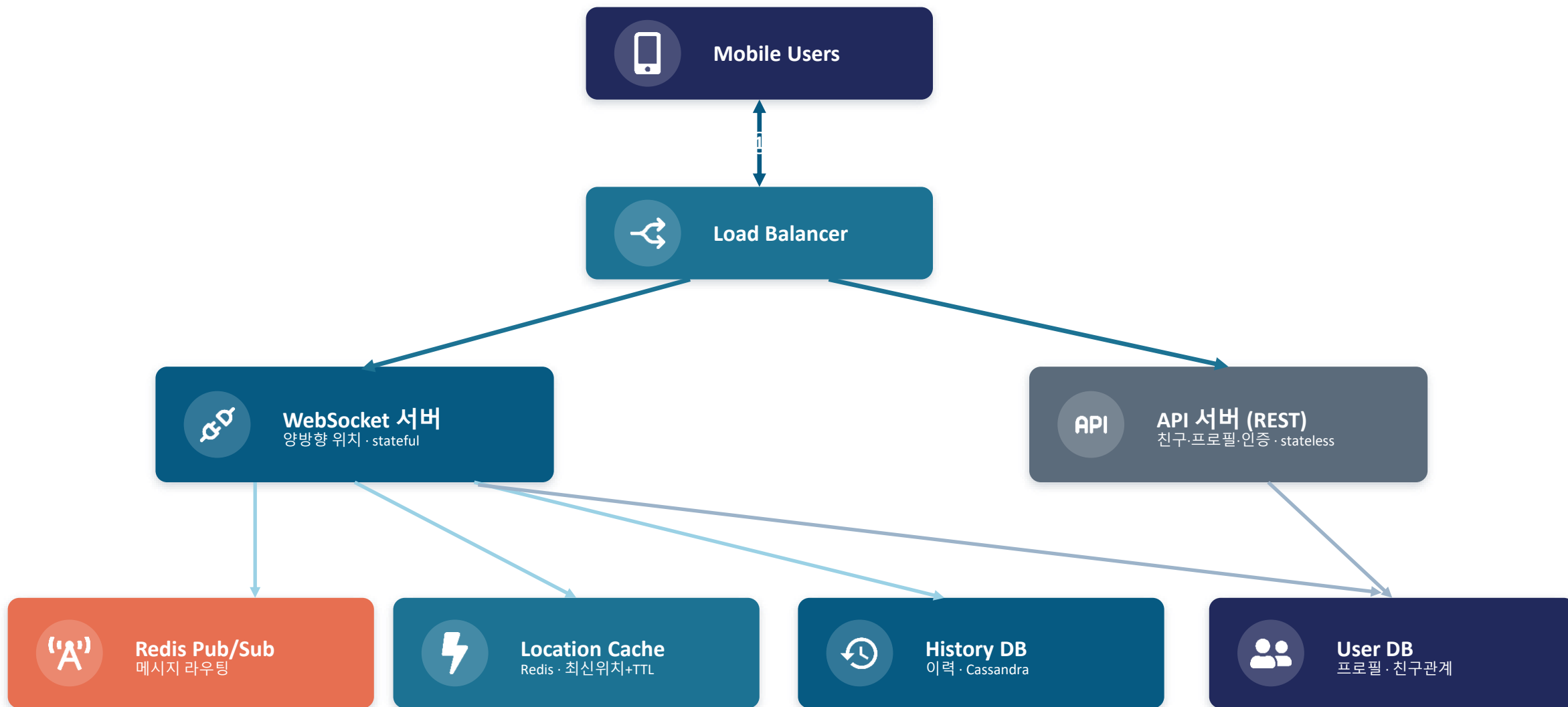
방법 B · 공유 백엔드



✓ 서버가 위치 수신 → 근처 친구에게만 전달

백엔드 3대 책임: ① 모든 위치 수신 ② 각 위치를 근처 친구에게 fan-out ③ 반경 밖이면 전달 안 함

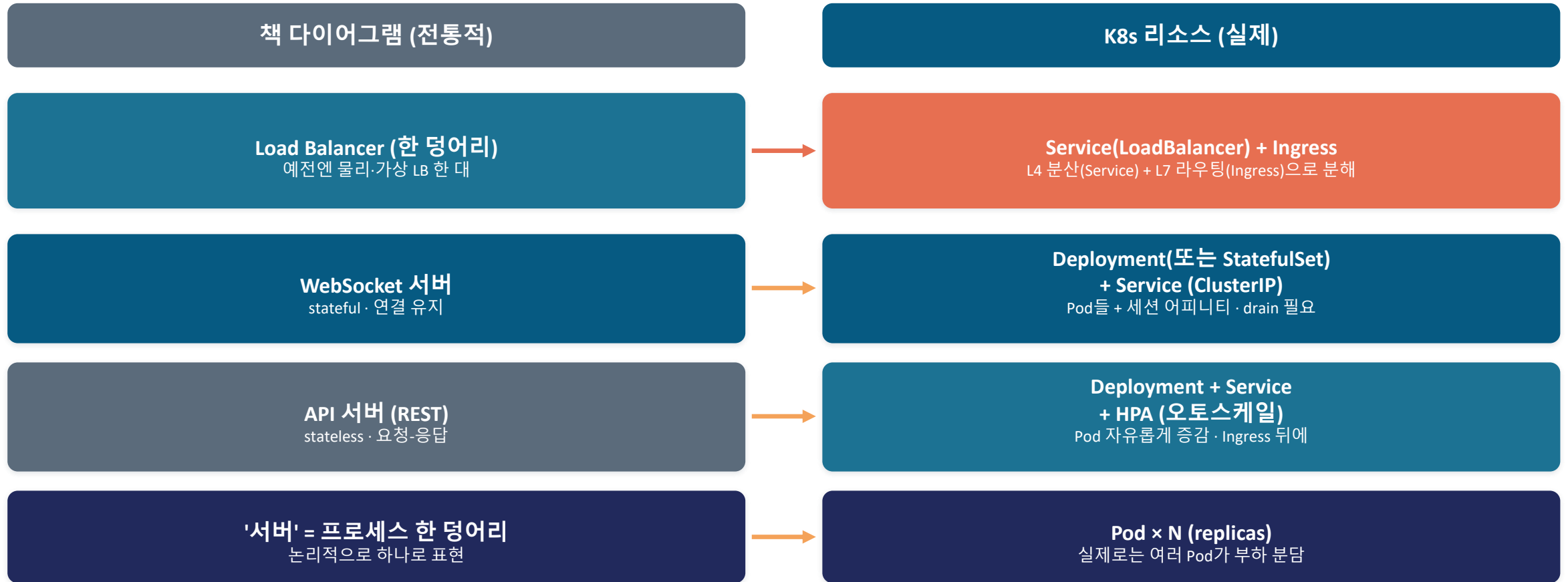
개략 아키텍처 (High-level Design)



2계층 서버: WebSocket(실시간·stateful) + REST API(일반 요청·stateless). 위치 데이터는 Redis 계열이 담당.

잠깐 — 이 그림은 '전통적' 표현입니다

책의 다이어그램은 LB·서버가 각각 한 덩어리. K8s 환경에선 여러 리소스로 분해된다(특히 LB가 추상화됨)

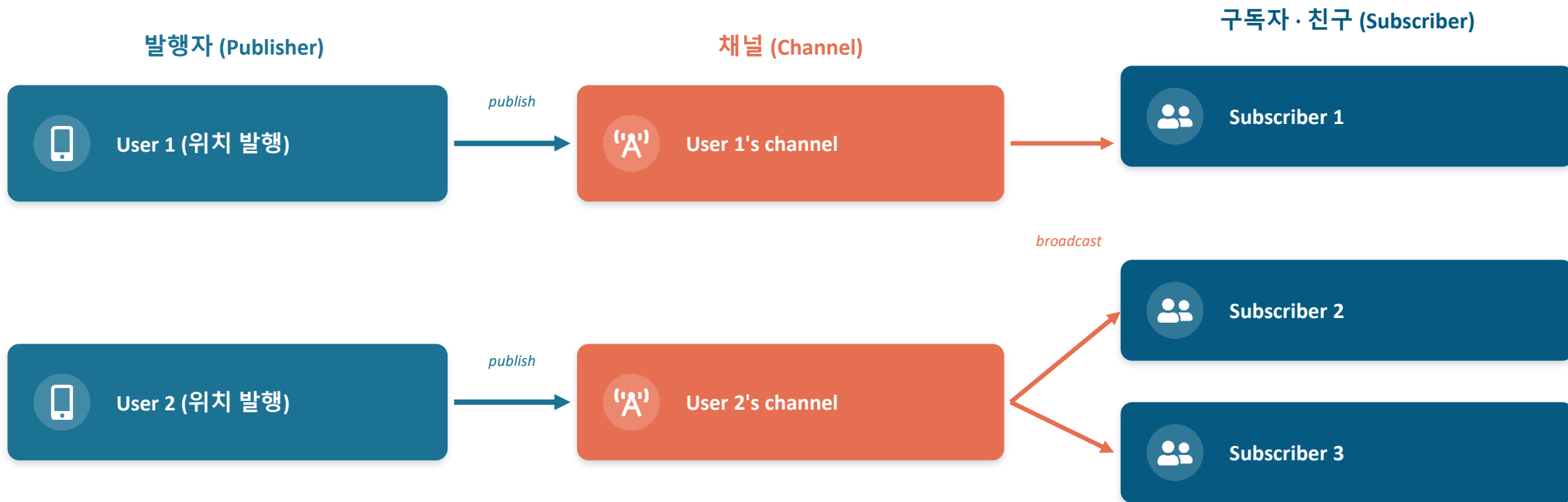


포인트: '박스 1개 = 리소스 1개'가 아니다. 특히 LB는 K8s에서 Service(L4)+Ingress(L7)로 흩어지고, 온프레임(RKE2)이면 그 앞을 MetalLB/kube-vip가 채운다.

컴포넌트별 역할 한눈에 보기

컴포넌트	역할
Load Balancer	REST 서버 + stateful WebSocket 서버 앞단에서 트래픽 분산
WebSocket 서버	친구 위치의 실시간 갱신 담당. 클라이언트당 1개 영속 연결. 초기화 시 근처 친구 위치 시딩
Location Cache (Redis)	활성 사용자의 최신 위치만 저장. TTL로 비활성 사용자 자동 삭제. 영속화 불필요
Redis Pub/Sub	위치 업데이트를 온라인 친구에게 라우팅하는 경량 메시지 버스. 채널 생성이 매우 저렴
User DB	사용자 프로필 + 친구 관계 저장. RDB 또는 NoSQL
Location History DB	위치 이력 저장 (ML용). 쓰기 부하 큼 → Cassandra, user ID로 샤딩

Redis Pub/Sub 은 어떻게 동작하나?

**핵심:**

사용자마다 '자기 채널'을 가짐. 위치 업데이트를 자기 채널에 발행(publish)하면, 그 채널을 구독(subscribe)하는 친구들이 받음. 구독자가 없으면 메시지는 그냥 버려짐 → 부하 거의 없음.

위치 업데이트가 흐르는 과정

- 1 클라이언트가 위치 업데이트를 LB로 전송
- 2 LB가 해당 클라이언트의 WebSocket 서버로 전달
- 3 WS 서버가 위치를 History DB에 저장
- 4 Location Cache에 최신 위치 갱신 (TTL 리셋)
- 5 사용자 '자기 채널'에 위치 발행 (3~5는 병렬 가능)
- 6 Pub/Sub이 채널 구독자(=온라인 친구들)에게 브로드캐스트
- 7 각 친구의 ws 핸들러가 거리 계산 → 반경 내면 전달, 아니면 드롭

★7단계의 '거리 계산 후 반경 필터링'이 fan-out의 실제 양을 400 → 약 40으로 줄이는 지점.

데이터 모델 & API

데이터 모델

Location Cache (Redis)

key	value
user_id	{lat, lng, timestamp}

Location History DB

user_id	lat	lng	ts
---------	-----	-----	----

→ user ID로 샵딩, 쓰기 부하 큼 → Cassandra 적합

왜 DB가 아닌 Redis?

'현재 위치'만 필요 → 사용자당 1건. 빠른 R/W + TTL 자동만료. 다운되면 새 캐시로 교체, 몇 사이클 놓쳐도 허용.

WebSocket API

① 주기적 위치 업데이트

클→서: lat, lng, ts / 응답 없음

② 위치 업데이트 수신

서→클: 친구 위치 + ts

③ WebSocket 초기화

클→서: 초기 위치 → 근처 친구 위치 수신

④ 친구 구독

친구 ID → 최신 위치 반환

⑤ 친구 구독 해제

친구 ID → 응답 없음

STEP 3

심화 설계 · Deep Dive

개략 설계는 대부분 잘 동작하지만, 우리 규모에선 깨진다.
컴포넌트를 하나씩 늘려가며 병목을 찾고 제거한다.

API 서버

WebSocket 서버

User DB

Location Cache

Redis Pub/Sub

Pub/Sub 대안(Erlang)

각 컴포넌트: *stateless*냐 *stateful* 이냐 → 스케일링 난이도가 갈린다.

서버 계층 스케일링

API 서버

STATELESS

- 상태가 없음 → 확장 쉬움
- CPU·부하·I/O 기준 오토스케일
- 노드 추가/제거 자유로움
- → 잘 알려진 표준 기법으로 충분

WebSocket 서버

STATEFUL

- 영속 연결 유지 → 확장 주의 필요
- 노드 제거 전 기존 연결 'drain' 필요
- LB에서 'draining' 표시 → 새 연결 차단
- → 좋은 LB가 stateful 오토스케일의 핵심

공통 원리: *stateless = 쉽게 확장 / stateful = 연결·데이터 이전을 신경 써야 함*

클라이언트 초기화 (앱 시작 시)

- 1 위치를 Location Cache에 갱신
- 2 연결 핸들러 변수에 위치 저장 (이후 거리 계산용)
- 3 User DB에서 내 친구 목록 전체 로드
- 4 Cache에 친구들 위치를 배치(batch)로 조회
- 5 각 친구와 거리 계산 → 반경 내 친구만 클라이언트로 반환
- 6 모든 친구(온·오프라인)의 Pub/Sub 채널 구독
- 7 내 현재 위치를 내 채널에 발행

팁: 오프라인 친구 채널도 미리 구독 → 친구가 온라인 될 때 별도 구독 처리 불필요 (설계 단순화, 메모리만 조금 더 사용)

Pub/Sub 서버는 몇 대 필요한가?

메모리 사용량

채널 수 = 1억 (10억 × 10%)

1억 × 20B × 100 친구 ≈ 200GB

100GB 서버 기준 → 약 2대

메모리는 병목 아님

CPU 사용량

전달 부하 = 1,400만 push/초

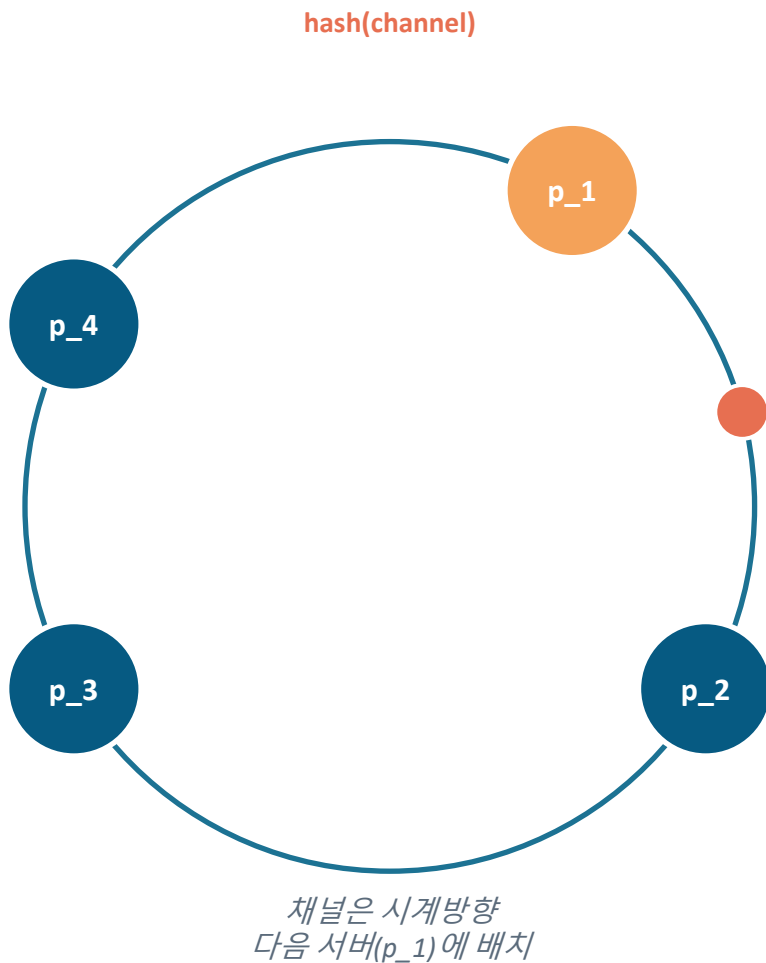
서버당 보수적 10만 push/초 가정

14M ÷ 100K ≈ 140대

CPU가 진짜 병목 ★

결론: 병목은 메모리가 아니라 CPU (전달 부하). → 분산 Redis Pub/Sub 클러스터가 필요하다.

분산 Pub/Sub · Consistent Hashing with Virtual Node



1

채널 = 서로 독립

publisher user ID로 샤딩 → 여러 서버에 분산

2

Service Discovery

etcd/ZooKeeper에 해시링 저장 (KV store)

3

ws 서버가 구독

해시링 변경 시 알림 받아 로컬 캐시 갱신

4

노드 교체는 저렴

해당 서버 채널만 이동 → 교체가 추가보다 안전

리사이징 시 채널이 대량 이동 → 재구독 폭주 + 일부 업데이트 유실. 트래픽 최저 시간대에 수행.

의문 — 친구가 많으면 채널도 폭발하지 않나?

“한 사람 친구가 몇 억이면, 그만큼 채널을 구독(sub)해야 하는 거 아냐? 너무 많을 것 같은데...”

✗ 오해: follower 모델

셀럽 1명을 수백만이 팔로우 → 한쪽이 수억 구독자를 가짐. 이러면 채널·구독이 폭발.

→ Nearby Friends는 이 모델이 아님

✓ 실제: friend 모델

친구는 양방향 + 하드 캡(예: FB 5,000명). 셀럽 팔로워 같은 무제한이 아님.

→ 한 사람이 구독하는 채널 = 최대 5,000개

구분	실제 숫자	부담
한 사람이 구독하는 채널	최대 5,000개 (평균 400)	사용자 쪽 · 미미
전체 채널 총량 (사용자당 1개)	약 1억 개	메모리 2대면 OK
진짜 병목	1,400만 push/초	CPU → ~140대 분산

whale(친구 5,000 짝 참)도 OK — 구독자가 여러 ws 서버에 흩어지고, 채널도 100+ Pub/Sub 서버에 분산돼 핫스팟 없음.

확장 · 랜덤 '근처 사람' (Geohash)

지역을 Geohash 격자로 분할, 격자마다 채널

9q8zn6	9q8znd	9q8znf
9q8zn3	9q8zn9 	9q8znc
9q8zn2	9q8zn8	9q8znb

중앙 격자 + 주변 8칸 구독 → 경계 문제 해결

동작 방식

- 위치 갱신 시 자기 geohash 계산
- 해당 geohash 채널에 발행
- 같은 격자 구독자(발신자 제외)가 수신

친구 기능과의 차이

채널 기준

친구: 사용자별 채널

랜덤: 위치(격자)별 채널

Redis Pub/Sub 의 대안 · Erlang

왜 Erlang인가?

- 고도 분산·동시성에 특화된 런타임(BEAM)
- 초경량 프로세스: 1개 $\approx$ 300 bytes
- 단일 서버에 수백만 프로세스 가능
- 유희 프로세스는 CPU 0 $\rightarrow$ 1천만 사용자를 각각 프로세스로 모델링
- 프로세스 간 구독이 언어에 내장(OTP)

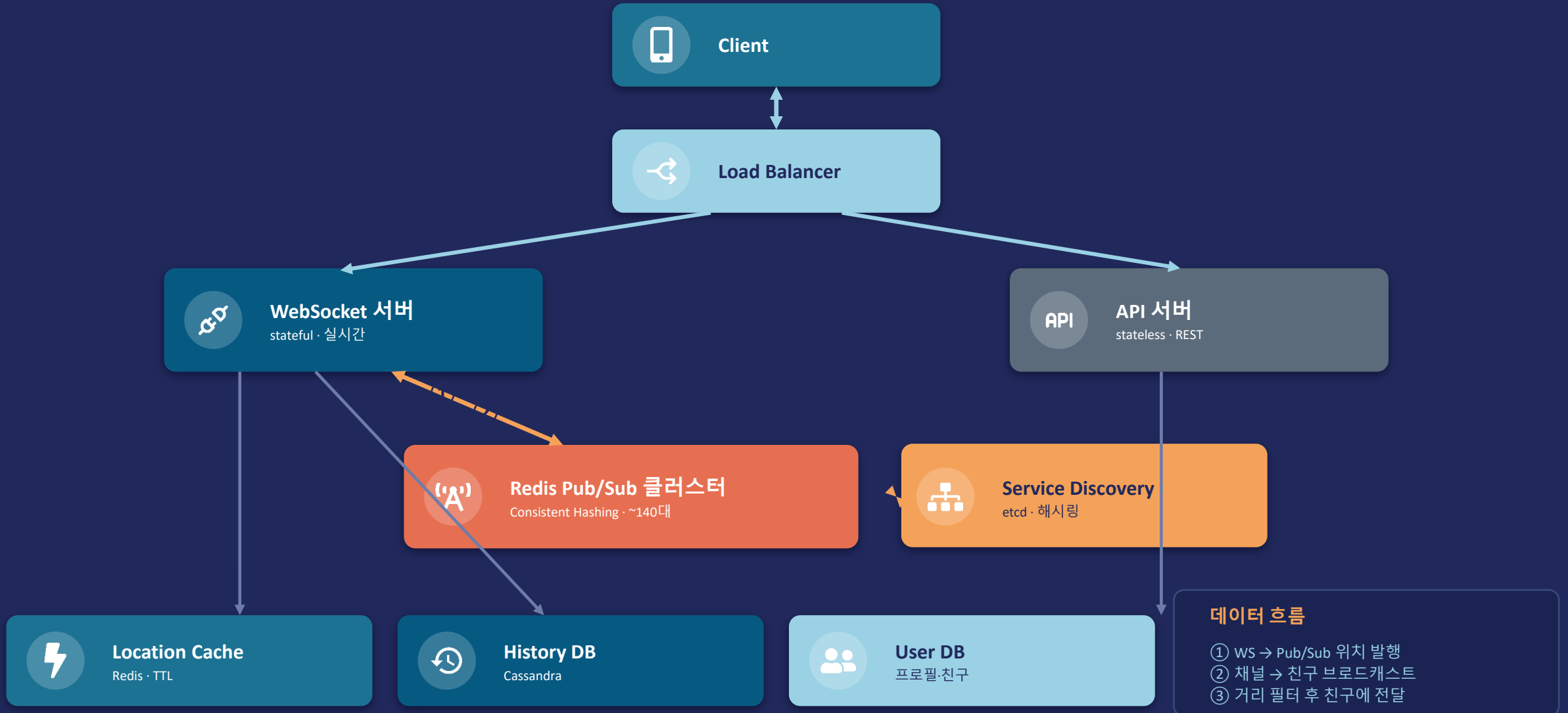
트레이드오프

각 사용자 = Erlang 프로세스.
WS 서버 + Pub/Sub 클러스터 전체를
분산 Erlang 앱으로 대체 가능.

단점: Erlang은 니치(niche).
좋은 개발자 채용이 어렵다.
팀에 역량 있을 때만 좋은 선택.

책의 결론: Erlang 이 이 문제엔 Redis Pub/Sub 보다 더 나은 해법일 수 있으나, 현실적 제약(인력)을 고려해야 함.

전체 아키텍처 한 장 종합



실시간(WS) · 캐시(Redis) · 라우팅(Pub/Sub) 3 축이 유기적으로 fan-out을 감당하는 구조.

한 페이지 정리

핵심 문제

친구의 '움직이는' 위치를 실시간으로 근처 친구에게 전파. 정적 색인 불가 → 메시지 전파가 본질.

규모

위치 업데이트 334K/s → fan-out 시 1,400만/s. 받기보다 뿌리기가 40배 부하.

핵심 구조

WebSocket(실시간 연결) + Location Cache(Redis-TTL) + Pub/Sub(사용자별 채널로 친구에 라우팅).

스케일 관건

Pub/Sub CPU가 병목 → Consistent Hashing으로 ~140대 분산. WS는 stateful이라 draining 필요.

30초 요약

위치 변경 → ws서버 수신 → 내 채널에 발행 → Pub/Sub이 친구 채널 구독자에 브로드캐스트 → 각 친구 서버가 거리 계산 → 반경 내면 전달

AI 시대라면, 이 설계는 어떻게 달라질까?



위치 예측 (ML)

History DB를 학습해 '다음 위치'를 예측하면? 업데이트 주기를 늘려 fan-out 부하를 줄일 수 있을까?



의미 기반 추천

'근처 친구'를 넘어 '지금 만나면 좋을 친구'를 추천하려면? 임베딩·벡터 검색을 어디에 끼울까?



이상 탐지·관측성

초당 1,400만 이벤트 스트림에서 이상 패턴(스푸핑, 봇)을 실시간 탐지하려면 어떤 파이프라인이 필요할까?



AI 에이전트가 소비

사람이 아닌 AI 에이전트가 이 위치 스트림을 구독한다면? 인증·권한·rate limit 설계는 어떻게 바뀔까?



LLM 실시간성 한계

LLM을 실시간 경로에 넣으면 지연이 문제. 어디는 사전계산(배치), 어디는 실시간으로 나눌까?



Thank you

Nearby Friends · 실시간 근처 친구 시스템 설계

Q & A

